Assignment – Part A: Individual Programming Exercise – Comparing Programming Languages

A1: Code Exploration

This section explores the tutor-provided Python and JavaScript scripts. These scripts examine a list of names, remove duplicates, and return them sorted alphabetically. The aim is to run both scripts, observe results, make one modification, and compare how the same task is expressed in each language. Google Colab was used to run and test the scripts because it provides a simple notebook environment for executing code and recording outputs. Python ran directly in a standard Colab cell. Since JavaScript is not the standard Colab execution runtime, the `%%writefile` command was used to save the JavaScript into a `.js` file and the `!node` command was used to execute it (see Appendix 1).

1.1 Original Script Exploration

The tutor-provided Python script (see Appendix 2) uses a function called `process_names(names)`. This is a user-defined function which expects a list of names as its input. Inside this function, `set(names)` is used to remove duplicate values. The resulting set is converted back into a list using `list()` so that the values can be sorted and returned in list form. The `.sort()` method is used to arrange the names, after duplicate values have been removed, in alphabetical order before the

processed list is returned. The test list provided in Figure 1 gives a list of names, some of which are duplicated but with inconsistent capitalisation:

```
test_names = ["Alice", "bob", "alice", "Charlie", "BOB", "dave", "Eve", "charlie"]
```

Figure 1: Python test_names

The script was tested using the `print()` function, which outputs both the original test list and the processed list returned by `process_names()`:

```
print("Original list:", test_names)
print("Processed list:", process_names(test_names))
```

Figure 2: Python print()

The results below show that the original list is printed as defined in `test_names`, while the list generated by the `process_names()` function has reordered the names in alphabetical order:

```
Original list: ['Alice', 'bob', 'alice', 'Charlie', 'BOB', 'dave', 'Eve', 'charlie']
Processed list: ['Alice', 'BOB', 'Charlie', 'Eve', 'alice', 'bob', 'charlie', 'dave']
```

Figure 3: Python test result

The output shows that the script removes exact duplicate values, but it does not remove names that an end user might reasonably view as duplicates. This is because the code is case-sensitive and only looks for exact string matches. For example, `'Alice'` and `'alice'`, and `'BOB'` and `'bob'` are visible in the test output. This is technically correct if the requirement is to remove exact string matches. However, it is a limitation where names are entered manually by users because inconsistent capitalisation is common in real datasets.

The tutor-provided JavaScript code (see Appendix 1) follows logic similar to the Python version but expresses it using JavaScript syntax. The `processNames(names)` function is used. It requires an array of names as its input. Inside the function, `new Set(names)` is used to remove duplicate values. The spread syntax `[...new Set(names)]` converts the set back into an array which is then sorted alphabetically using `.sort()`. The JavaScript code was tested using the same list of names as the Python code, with `console.log()` used to display the original and processed arrays. The results produced the same logical output as the Python script, generating an array rather than a Python-style list:



Figure 4: JavaScript test result

This confirms that both the original scripts behave in the same way. They remove exact duplicates and sort the remaining values; however, they do not identify duplicates where capitalisation differs. As such, the same limitation is present in both scripts.

Since the tutor-provided scripts treat differently capitalised names as separate values, both scripts were modified to make duplicate removal case-insensitive (e.g., `'Alice'` and `'alice'` are considered the same value).

1.2 Modified Script and Testing

Both scripts were modified so duplicate removal ignores case, while keeping the name as it was first entered rather than converting everything to one case. This distinguishes the value used for comparison from the value for output. The modified scripts (see Appendix 3 and Appendix 4) also include inline comments explaining the purpose of the lowercase comparison key, the tracking set, and the preservation of the first-entered name. This supports readability and maintainability because another developer can understand both the code and the reasons for the modification.

In the Python script (see Appendix 3 for full script modification), `set(names)` was replaced with a `for` loop, a `seen_lowercase` set was introduced to check whether a lowercase version of a name has already been encountered, and a `unique_names` list to preserve the first-entered version of each name:

```
seen_lowercase = set() # tracks the names in lowercase that have been kept
unique_names = []  # holds de-duped names while preserving original case

for name in names:
  key = name.lower()  # Creates a case insensitive comparison key

  # Keep this name only if a case-insenitivity match has not been observed
  # yet
  if key not in seen_lowercase:
    seen_lowercase.add(key)
    unique_names.append(name)
```

Figure 5: Python script modification

Within this loop, `.lower()` creates a lowercase comparison key for duplicate checking. Sorting was updated to `unique_names.sort(key=str.lower)` to transform each element once rather than repeatedly comparing pairs. The output below confirms that case de-duplication has been successful while the first occurrence of each name has been preserved:

```
Original list: ['Alice', 'bob', 'alice', 'Charlie', 'BOB', 'dave', 'Eve', 'charlie']
Processed list: ['Alice', 'bob', 'Charlie', 'dave', 'Eve']
```

Figure 6: Modified Python script output

In the JavaScript code (see Appendix 4 for full script modification), the same logic used in the modified Python script was applied using `seenLowercase` (a `Set`) and a `uniqueNames` array:

```
  const seenLowercase = new Set (); // tracks which lowercase names have been
                                    // kept
  const uniqueNames = [] // holds de-duped names while preserving original case

  for (const name of names) {
    const key = name.toLowerCase(); // creates a case-insensitive comparison key

    // Keep this name only if a case-insensitive match has not been seen yet
    if (!seenLowercase.has(key)) {
      seenLowercase.add(key);
      uniqueNames.push(name);
    }
}
```

Figure 7: JavaScript modification

`.toLowerCase()` is similar to Python's `.lower()` functionality; however, JavaScript requires a longer comparator function to achieve the same result:

```
// Sorts alphabethically, case-insensitive
uniqueNames.sort((a, b) => a.toLowerCase().localeCompare(b.toLowerCase()));
```

Figure 8: Modified JavaScript comparator function

The test output below confirms that the modified JavaScript function removes duplicate

values and preserves the first occurrence of each name, producing the same result as the modified Python script:

```
!node cleanNames_Modified.js

Original list: [
  'Alice', 'bob',
  'alice', 'Charlie',
  'BOB',   'dave',
  'Eve',   'charlie'
]
Processed list: [ 'Alice', 'bob', 'Charlie', 'dave', 'Eve' ]
```

Figure 9: Modified JavaScript test output

The modifications to the Python and JavaScript scripts resolve the case-sensitive issue present in the tutor-provided scripts. However, the modified scripts have two main limitations, both of which are common real-world data-cleaning issues:

1. They do not trim whitespace. For example, `'Alice'` and `'Alice '` would still be classified as different names.
2. They assume all values are strings; a `None` value in Python or a `null` value in JavaScript would cause a runtime error.

1.3 Modified Script Comparison

Both scripts share the same approach but differ in syntax. Python's `for` loop and `if key not in seen_lowercase` read almost like plain English which supports Downey's (2012, Chapter 1) view that Python is designed to be readable and accessible. JavaScript is also readable but its braces, semicolons and method calls, such as `.has()` and `.push()`, make it more syntactically explicit and slightly more verbose. The sorting logic also differs between the two languages. Python sorts case-insensitively using a single keyword argument,

`unique_names.sort(key=str.lower)`, whereas JavaScript requires a comparator function, `uniqueNames.sort((a, b) => a.toLowerCase().localeCompare(b.toLowerCase()))`, to achieve the same result. Using the selected happy-path test data (see Appendices 3 and 4), both modified scripts produced identical results with no runtime errors. The only output difference was formatting: Python displayed a list, while Node.js displayed a JavaScript array. A further output difference emerged once invalid input was introduced (see Appendices 5–8): Python and JavaScript raised differently named exceptions for the same underlying fault, a distinction explored further in A2's discussion of error handling.

[1,082 words]

A2: Language Comparison Report

The code exploration in A1 shows that Python and JavaScript can solve the same task, but they express the solution differently. This section compares the two languages by programming model, readability, error handling and suitability for modern computing tasks.

2.1 Programming Model

Both Python and JavaScript are high-level languages that can support procedural, object-oriented and functional programming styles. Guttag (2016, pp. 7-8) explains that high-level languages use more abstract operations than low-level languages and describes Python as a general-purpose language that is relatively simple and easy to learn. This supports Python's suitability for readable introductory programming tasks.

Python can be used procedurally through functions and step-by-step logic, in an object-oriented way through classes, and functionally through features such as higher-order functions and lambda expressions. JavaScript can also be used across different programming styles, but its functional features are especially important because functions are first-class values. As Flanagan (2020, Chapter 8) explains, functions can be assigned to variables, passed as arguments and returned from other functions. This is important in web-based applications because event handlers and callbacks often rely on functions being passed into other code.

In A1, both scripts were used procedurally. The Python function `process_names(names)` receives a list, stores lowercase comparison keys in a set and returns a sorted list (see Figure 10).

```python
    seen_lowercase = set() # tracks the names in lowercase that have been kept
    unique_names = []  # holds de-duped names while preserving original case

    for name in names:
      key = name.lower()  # Creates a case insensitive comparison key

      # Keep this name only if a case-insenitivity match has not been observed
      # yet
      if key not in seen_lowercase:
        seen_lowercase.add(key)
        unique_names.append(name)
```

Figure 10: Modified Python script process_names(names)

JavaScript's `processNames(names)` follows the same model, looping through an array and storing keys in a `Set` (see Figure 11).

```javascript
function processNames(names) {
  const seenLowercase = new Set (); // tracks which lowercase names have been
                                    // kept
  const uniqueNames = [] // holds de-duped names while preserving original case

  for (const name of names) {
    const key = name.toLowerCase(); // creates a case-insensitive comparison key

    // Keep this name only if a case-insensitive match has not been seen yet
    if (!seenLowercase.has(key)) {
      seenLowercase.add(key);
      uniqueNames.push(name);
```

Figure 11: Modified JavaScript code processNames(names)

In both cases, a function receives input, executes instructions step by step, and returns a result. This procedural approach suits the simple input-output structure of the task, whereas a more complex object-oriented design would add unnecessary maintenance overhead without improving the solution.

2.2 Readability and Developer Accessibility

Readability affects how efficiently source code can be understood, reviewed and maintained. Since software is rarely maintained by its original author, clarity is important for long-term maintainability. In A1, both Python and JavaScript are readable, but they express the same algorithm with different levels of concision and syntactic detail.

Python is more compact because it uses indentation rather than explicit block delimiters. Statements such as `if key not in seen_lowercase` read almost like natural English, making the logic easier to follow. Similarly, `unique_names.sort(key=str.lower)` is concise because Python's sorting method accepts a key function directly. This suits A1's short scripts where the main aim is to show the duplicate-removal logic clearly. JavaScript is also understandable, but more syntactically explicit. Buse and Weimer's (2010) empirical work on code readability is relevant because it links readability to measurable source-code features, including density of operators, punctuation and structural markers. In the JavaScript code, curly braces and method calls such as `.has()` and `.push()` create a denser layer of visual grammar than the Python version.

For A1, Python has the stronger readability advantage: its structure allows the reader to follow the duplicate-removal algorithm with minimal distraction. JavaScript requires more visual parsing, but this is not necessarily a general weakness. For example, `localeCompare()` makes locale-aware string comparison explicit, which can be useful in professional web-development contexts. The comparison therefore shows a

trade-off rather than a universal winner: Python's clarity suits concise, self-contained scripts, while JavaScript's explicitness can pay off where greater control over browser-based or internationalised behaviour is needed.

2.3 Error Handling and Debugging

Both languages support structured exception handling. Haverbeke (2024, Chapter 8) explains that JavaScript can respond to a failing function by returning a special value such as `null`, or by throwing an exception handled by `try`/`catch`. Testing the modified JavaScript code with `null` produced an unhandled `TypeError: Cannot read properties of null (reading 'toLowerCase')` (see Appendix 5). The JavaScript stack trace also supported debugging by showing where the `toLowerCase()` failure occurred, although this was slightly less direct in Colab because the script first had to be written to a `.js` file and executed with `!node`. Wrapping the call in `try`/`catch` allowed the error to be caught without the script failing unexpectedly (see Appendix 6).

Python also uses exceptions to handle runtime errors. The Python Software Foundation (2026, section 8.3) explains that `try`, `except`, `else`, and `finally` clauses can be used for structured exception handling. Without this, Python produces a traceback showing the error's line and type, which helps direct the developer to the cause. The equivalent Python test produced an `AttributeError`, since `NoneType` has no `.lower()` method (see Appendix 7). Rather than modifying the function itself, the

function call was placed inside a `try`/`except`/`else`/`finally` block, allowing the `AttributeError` to be handled without the script stopping unexpectedly (see Appendix 8). This is a reactive approach: the error is allowed to occur and is then managed, rather than being prevented beforehand.

Validating inputs before removing duplicates would be the stronger approach. Failure to validate inputs has business implications, since an unhandled `None` or `null` could delay a report or disrupt processing, while inadequately handled invalid data could produce unreliable downstream outputs. Both languages therefore need the same discipline around input validation.

2.4 Suitability for Modern Computing Tasks

Python and JavaScript are suited to different modern computing tasks. JavaScript is suited to web applications, running directly in the browser to add interactivity. It can support server-side development through `Node.js` (Tilkov and Vinoski, 2010). This dual reach makes JavaScript a natural fit for full-stack development, where the same language can support client-side interactivity and server-side logic. JavaScript-focused projects centre on user interaction, real-time behaviour and web-based reach.

In contrast, Python is typically used for data processing, automation and artificial intelligence. These uses are supported by a data science ecosystem that includes *pandas* for data manipulation and *scikit-learn* for machine learning. Pedregosa *et al*. (2011) explain that *scikit-learn* emphasises ease of use, performance, documentation

and API consistency. This makes Python well suited to backend logic, statistical analysis and model training, where the priority is analysing data rather than presenting it to a user directly.

Language suitability is therefore a deployment decision as much as a technical one. If the A1 duplicate-removal task formed part of a back-office data-quality pipeline, Python's readability and data ecosystem would make it the stronger choice. If the same logic needed to run inside a customer-facing web form, JavaScript's ability to validate input in the browser, without a round-trip to a server, would make it the more practical option. Language choice therefore shapes not only how code is written, but where it can run and who maintains it.

[1,074 words]

A3: Professional Reflection

This coding exercise shows that even simple scripts can raise important professional, ethical and business issues. The tutor-provided Python and JavaScript scripts remove duplicate names and sort a list, but they do not fully consider wider issues such as data quality and software design. In professional settings, code should not only run successfully; it should be readable, maintainable, tested and suitable for the context in which it is used.

3.1 The Importance of Readable and Ethical Code

Readable code is important because software is rarely maintained by a single software engineer. Therefore, businesses need clear governance across the software development lifecycle, including post-production maintenance support. The Software Engineering Body of Knowledge (SWEBOK) identifies the core principles that support readable and maintainable code, including reducing unnecessary complexity, planning for change, building code that can be tested and verified, reusing existing components where appropriate, and following recognised construction standards (Washizaki, 2025, Chapter 4). These principles are relevant to the exercise in A1. The modified scripts use meaningful variable names such as `unique_names` and `seen_lowercase`. The inline comments explain why the comparison is case-insensitive, rather than simply describing each line. This is illustrated in the excerpt below (see Appendix 3 for the full script):

```
def process_names(names):
# Code modifed to dedupe on a case-insensitive basis, while keeping the first
# version of the name encountered ensuring the original encountered
# capitalisation is maintained rather than forcing the outputs into a single
# case.
    seen_lowercase = set() # tracks the names in lowercase that have been kept
    unique_names = []  # holds de-duped names while preserving original case
```

Figure 12: Modified Python script inline comments

Ethical coding is important because data represents real people. Names may appear simple, but once linked to accounts or records, they become personal data. The tutor-provided scripts treated `'Alice'` and `'alice'` as different values due to case-sensitive matching. In a computer system, this could create duplicate customer records, leading to repeated communications, inaccurate reporting or poor customer service. Conversely, over-aggressive merging could wrongly link two different people.

The wider consequences of poor data handling and inadequate testing are illustrated by the TSB platform migration, which caused some customers to see other users' account balances and transaction histories (Edser and Peachey, 2022). Although not the same type of coding issue as the name-processing scripts in A1, it illustrates the broader professional risk of poor data handling, inadequate testing and weak change control. The example reinforces the need for reliable identifiers, careful testing and strong governance when software interacts with customer records.

Working through this tension in my own code modification in A1 changed my initial view that resolving case-sensitivity was a straightforward improvement. There is no single correct approach for identifying duplicates; the appropriate approach depends on business context and the cost of each type of error. Professionally, I would treat this as

a decision requiring governance or compliance input, not one that a developer should make in isolation.

3.2 Bias, Privacy and Misuse in Programming

Bias can emerge from small coding choices. Walton (2024) suggests that computing systems are shaped by the social and cultural environments in which they are created, meaning they cannot be considered entirely neutral. This is relevant to the modified scripts from A1: although treating differently capitalised names as duplicates improves the original logic, name handling is culturally sensitive. Names may contain accents, hyphens, apostrophes or prefixes that do not fit simple assumptions. Data cleaning remains challenging in real-world contexts because datasets frequently include inconsistent entries, formatting errors and duplicate records, particularly where data from multiple sources causes the same entity to be represented inconsistently across records (Rahm and Do, 2000; Guo et al., 2023).

In banking technology environments, this is a familiar problem, as customer data is frequently integrated across multiple internal platforms. Writing the case-insensitivity fix initially felt like a complete solution. However, considering non-Latin and culturally varied naming conventions changed that view; I now see it as one of several fixes that a real system would need, not a general answer to matching names correctly.

Privacy is another common issue. The exercise in A1 uses synthetic test data, so printing the names in a list does not pose a risk. In a production system, however, the

same testing would write real customer names into system logs. This data could then be retained, copied, or accessed by people who do not need it. If retaining these names was unnecessary, this could conflict with the GDPR data minimisation principle, which requires personal data to be adequate, relevant and limited to what is necessary for the purpose of processing (Regulation (EU) 2016/679, 2016, art. 5(1)(c)).

Misuse is also possible. A name-cleaning script built for data quality could equally be repurposed for identity matching or profiling without an appropriate lawful basis or adequate human oversight, which could contribute to unfair decisions. For example, inconsistent name-matching in a credit scoring pipeline could cause an applicant's records to be incorrectly merged with, or separated from, another individual's history, distorting their assessed creditworthiness and potentially contributing to financial exclusion. Professional codes of conduct are in place to ensure computing professionals consider the wider impact of their work and to safeguard public interest (ACM, 2018; BCS, 2022). These codes apply to short scripts and large systems alike, since small assumptions can scale into operational or ethical problems.

The limitations identified in A1 also pose an operational problem. Untrimmed whitespace or unvalidated input types are minor in a classroom script, but the same assumptions in a larger system could distort customer records or downstream reporting. This changed how I view testing. In A1, I mainly used expected-input (“happy-path”) test scenarios. However, edge cases such as `None`, `null`, blank strings and unexpected input types are just as important for testing the robustness of code.

3.3 Real-World Impact

The 2012 RBS, NatWest and Ulster Bank IT incident illustrates the impact of poor software development practices at scale. The FCA fined the banks £42 million after a failed batch-processing update and unsuccessful attempt to reverse it disrupted payment services for millions of customers, and the PRA fined a further £14 million for inadequate systems and controls (Financial Conduct Authority, 2014; Bank of England, 2014). This links back to the testing limitations identified in A1: inadequate testing is not merely a coding exercise concern, but a cause of genuine, large-scale operational failure. This incident demonstrates that clear documentation, readable code, input validation, systematic testing and controlled deployment are not merely classroom concerns. In production systems, weaknesses in testing and change control can develop into operational-resilience failures affecting customers at scale.

This exercise has shown me that professional programming is not simply about producing working code. My key learning is that computing professionals must make their assumptions visible, test deliberately rather than conveniently, and consider how small coding choices may affect people, organisations and public trust.

[1,078 words]

References

ACM (2018) *ACM Code of Ethics and Professional Conduct*. New York: Association for Computing Machinery. Available at: https://www.acm.org/binaries/content/assets/about/acm-code-of-ethics-booklet.pdf (Accessed: 05 July 2026).

Bank of England (2014) *PRA fines Royal Bank of Scotland, Natwest Bank and Ulster Bank £14 million for IT failures*. Available at: https://www.bankofengland.co.uk/news/2014/november/pra-fines-rbs-natwest-and-ulster-bank (Accessed: 05 July 2026).

BCS, The Chartered Institute for IT (2022) *Code of Conduct for BCS Members*. Version 8. Available at: https://www.bcs.org/media/2211/bcs-code-of-conduct.pdf (Accessed: 05 July 2026).

Buse, R.P.L. and Weimer, W.R. (2010) 'Learning a metric for code readability', *IEEE Transactions on Software Engineering,* 36(4), pp. 546–558. Available at: https://doi.org/10.1109/TSE.2009.70

Downey, A.B. (2012) *Think Python*. Sebastopol, CA: O'Reilly. Available at: https://learning.oreilly.com/library/view/think-python/9781449332006/index.html (Accessed: 03 July 2026).

Edser, N. and Peachey, K. (2022) 'UK bank fined £49m over IT system meltdown', *BBC News*, 20 December. Available at: https://www.bbc.co.uk/news/business-64036529 (Accessed: 04 July 2026).

Financial Conduct Authority (2014) *FCA fines RBS, NatWest and Ulster Bank Ltd £42 million for IT failures*. Available at: https://www.fca.org.uk/news/press-releases/fca-fines-rbs-natwest-and-ulster-bank-ltd-%C2%A342-million-it-failures (Accessed: 05 July 2026).

Flanagan, D. (2020) *JavaScript: The Definitive Guide*. 7th edn. Sebastopol, CA: O'Reilly Media, Inc. Available at: https://learning.oreilly.com/library/view/javascript-the-definitive/9781491952016/ (Accessed: 06 July 2026).

Guo, M. *et al.* (2023) 'Normal workflow and key strategies for data cleaning toward real-world data: viewpoint', *Interactive Journal of Medical Research*, 12, e44310. Available at: https://doi.org/10.2196/44310

Guttag, J.V. (2016) I*ntroduction to Computation and Programming Using Python: With Application to Understanding Data*. 2nd edn. Cambridge, MA: The MIT Press.

Haverbeke, M. (2024) *Eloquent JavaScript*. 4th edn. Available at: https://learning.oreilly.com/library/view/eloquent-javascript-4th/9781098182502/ (Accessed: 06 July 2026).

Pedregosa, F. *et al.* (2011) 'Scikit-learn: Machine learning in Python', *Journal of Machine Learning Research*, 12, pp. 2825–2830. Available at: https://jmlr.org/papers/volume12/pedregosa11a/pedregosa11a.pdf (Accessed: 06 July 2026).

Python Software Foundation (2026) '8. Errors and Exceptions', *The Python Tutorial*. Available at: https://docs.python.org/3/tutorial/errors.html (Accessed: 06 July 2026).

Rahm, E. and Do, H.H. (2000) ‘Data cleaning: Problems and current approaches’, *IEEE Data Engineering Bulletin*, 23(4), pp. 3-13. Available at: https://www.researchgate.net/publication/220282831_Data_Cleaning_Problems_and_Current_Approaches (Accessed: 04 July 2026).

Regulation (EU) 2016/679 of the European Parliament and of the Council (General Data Protection Regulation) (2016) *Official Journal L 119*, 4 May, pp. 1–88. Available at: https://www.legislation.gov.uk/eur/2016/679/article/5 (Accessed: 05 July 2026).

Tilkov, S. and Vinoski, S. (2010) ‘Node.js: Using JavaScript to build high-performance network programs’, *IEEE Internet Computing*, 14(6), pp. 80–83. Available at: https://doi.org/10.1109/MIC.2010.145

Walton, D.J. (2024) *Culturally Responsive Computing: An Introduction into Computer Science, Security, and Technology*. Haverhill, MA: ROTEL. Available at: https://rotel.pressbooks.pub/culturally-responsive-computing/ (Accessed: 02 July 2026).

Washizaki, H. (ed.) (2025) *Guide to the Software Engineering Body of Knowledge v4.0a*. Los Alamitos, CA: IEEE Computer Society. Available at: https://computer.org/swebok (Accessed: 04 July 2026).

Appendices

Appendix 1: CleanNames JavaScript

The `%%writefile` and `!node` commands were used to execute the code in a Google Colab Notebook.



```
%%writefile cleanNames.js

function processNames(names) {
    let uniqueNames = [...new Set(names)]; // removes duplicates
    uniqueNames.sort(); // sorts alphabetically
    return uniqueNames;
}

// Test list with duplicates and inconsistent capitalisation
const testNames = ["Alice", "bob", "alice", "Charlie", "BOB", "dave", "Eve", "charlie"];

console.log("Original list:", testNames);
console.log("Processed list:", processNames(testNames));
```

```
Writing cleanNames.js
```

```
!node cleanNames.js
```

```
Original list: [
  'Alice', 'bob',
  'alice', 'Charlie',
  'BOB',   'dave',
  'Eve',   'charlie'
]
Processed list: [
  'Alice',   'BOB',
  'Charlie', 'Eve',
  'alice',   'bob',
  'charlie', 'dave'
]
```

Link to evidence: https://colab.research.google.com/drive/17GIB9DLZ_nEyJt5DbWUp6gBMPf8ga56j?usp=share_link

Appendix 2: Tutor-Provided Python Script (clean_names.py)

The unmodified tutor-provided Python script.



```
[1]
✓ 0s
# clean_names.py

def process_names(names):
    unique_names = list(set(names))  # removes duplicates but doesn't preserve order
    unique_names.sort()  # sorts alphabetically
    return unique_names

# Test list with duplicates and inconsistent capitalisation
test_names = ["Alice", "bob", "alice", "Charlie", "BOB", "dave", "Eve", "charlie"]

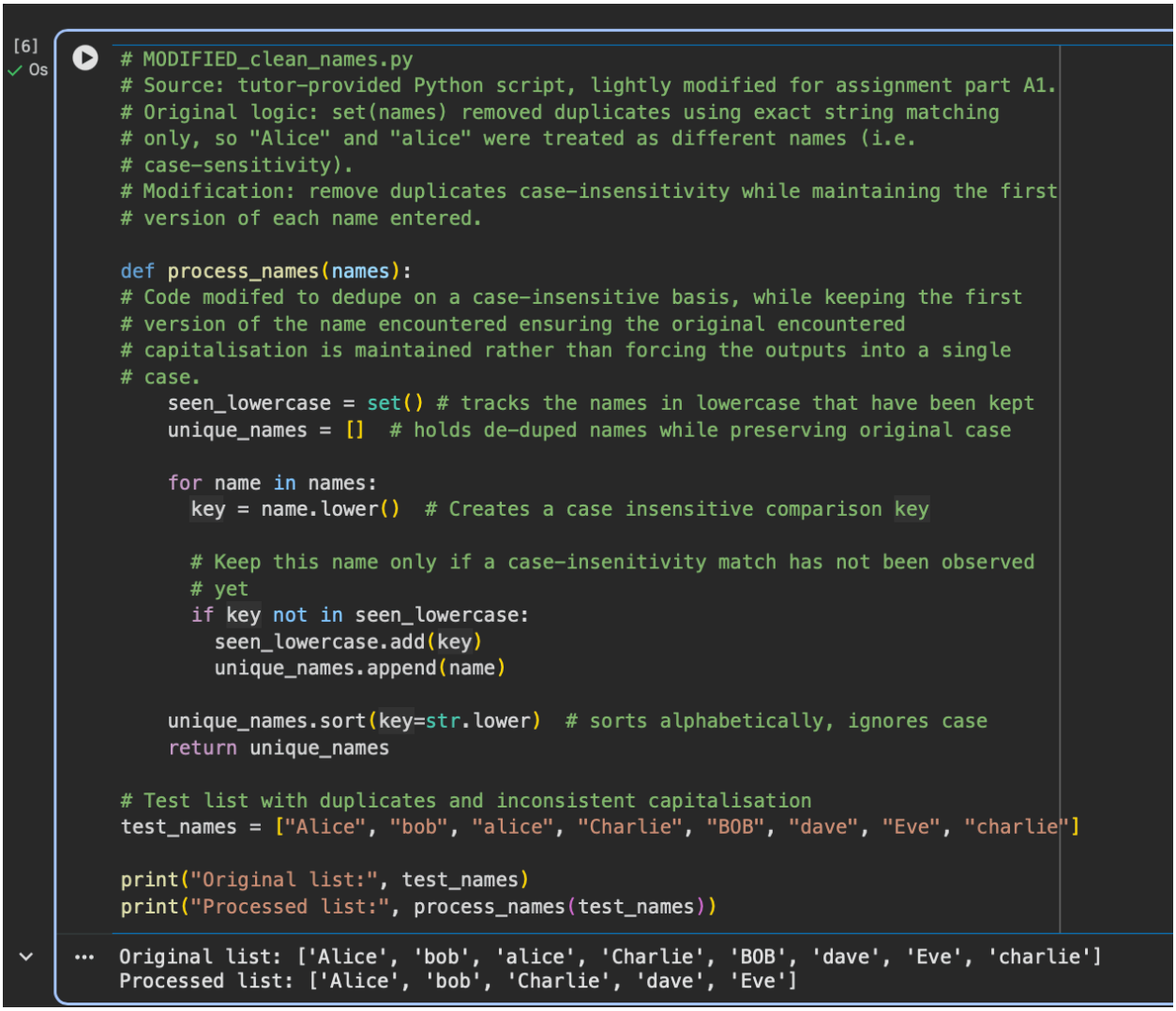
print("Original list:", test_names)
print("Processed list:", process_names(test_names))

Original list: ['Alice', 'bob', 'alice', 'Charlie', 'BOB', 'dave', 'Eve', 'charlie']
Processed list: ['Alice', 'BOB', 'Charlie', 'Eve', 'alice', 'bob', 'charlie', 'dave']
```

Link to evidence: https://colab.research.google.com/drive/1sc-Ye6VOEwp2zz-Y3mnfHLZloS4qEG2T?usp=share_link

Appendix 3: Modified Python Script

The script was modified to become case-insensitive.



```
[6]
✓ 0s
# MODIFIED_clean_names.py
# Source: tutor-provided Python script, lightly modified for assignment part A1.
# Original logic: set(names) removed duplicates using exact string matching
# only, so "Alice" and "alice" were treated as different names (i.e.
# case-sensitivity).
# Modification: remove duplicates case-insensitivity while maintaining the first
# version of each name entered.

def process_names(names):
# Code modifed to dedupe on a case-insensitive basis, while keeping the first
# version of the name encountered ensuring the original encountered
# capitalisation is maintained rather than forcing the outputs into a single
# case.
    seen_lowercase = set() # tracks the names in lowercase that have been kept
    unique_names = []  # holds de-duped names while preserving original case

    for name in names:
      key = name.lower()  # Creates a case insensitive comparison key

      # Keep this name only if a case-insenitivity match has not been observed
      # yet
      if key not in seen_lowercase:
        seen_lowercase.add(key)
        unique_names.append(name)

    unique_names.sort(key=str.lower)  # sorts alphabetically, ignores case
    return unique_names

# Test list with duplicates and inconsistent capitalisation
test_names = ["Alice", "bob", "alice", "Charlie", "BOB", "dave", "Eve", "charlie"]

print("Original list:", test_names)
print("Processed list:", process_names(test_names))

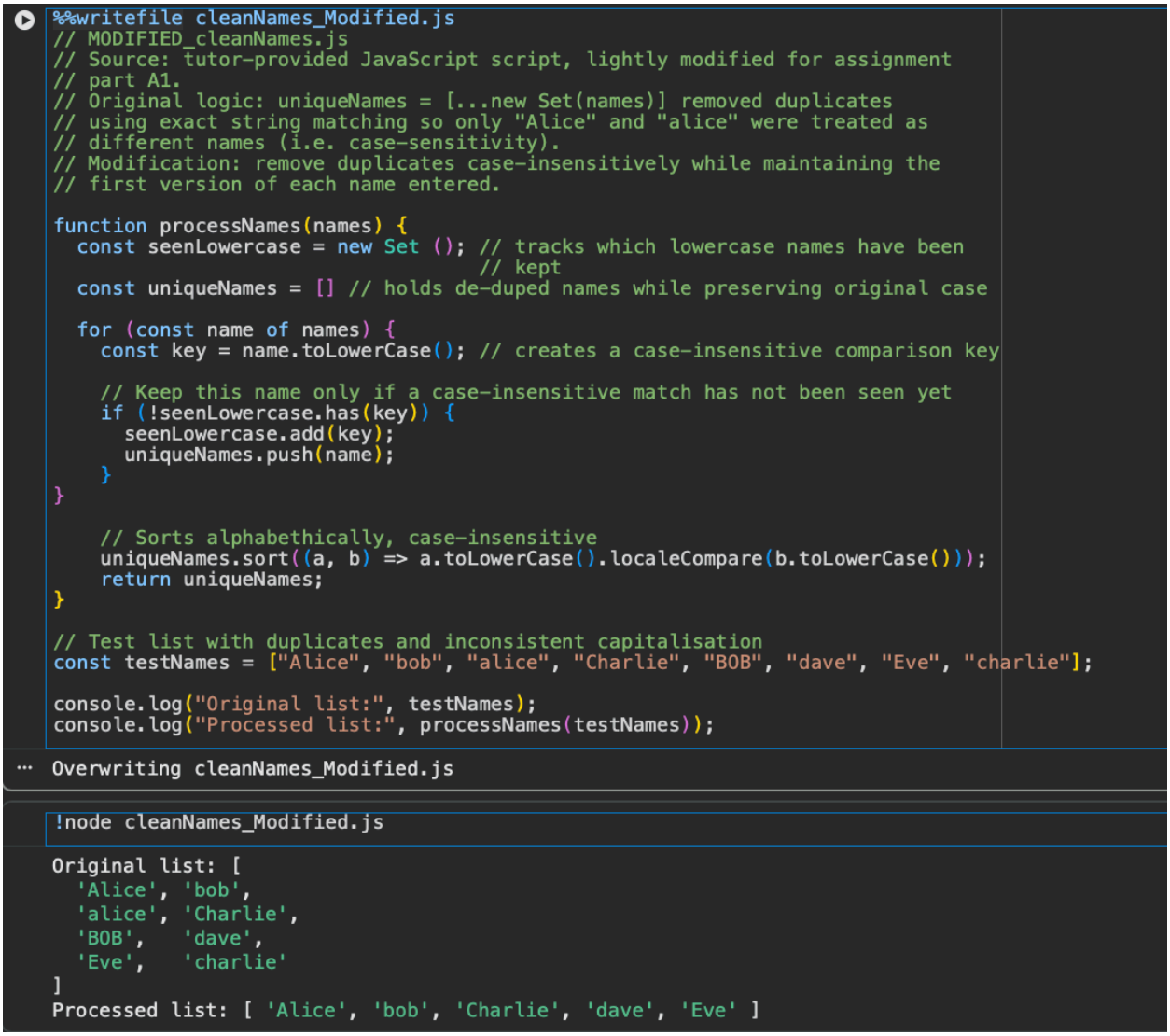
Original list: ['Alice', 'bob', 'alice', 'Charlie', 'BOB', 'dave', 'Eve', 'charlie']
Processed list: ['Alice', 'bob', 'Charlie', 'dave', 'Eve']
```

Link to evidence: https://colab.research.google.com/drive/

12kZOVbl7FV5Dpn_IAQyDHJX4-iQiqFwA?usp=share_link

Appendix 4: Modified JavaScript code

The script was modified to become case-insensitive.



```
%%writefile cleanNames_Modified.js
// MODIFIED_cleanNames.js
// Source: tutor-provided JavaScript script, lightly modified for assignment
// part A1.
// Original logic: uniqueNames = [...new Set(names)] removed duplicates
// using exact string matching so only "Alice" and "alice" were treated as
// different names (i.e. case-sensitivity).
// Modification: remove duplicates case-insensitively while maintaining the
// first version of each name entered.

function processNames(names) {
  const seenLowercase = new Set (); // tracks which lowercase names have been
                                    // kept
  const uniqueNames = [] // holds de-duped names while preserving original case

  for (const name of names) {
    const key = name.toLowerCase(); // creates a case-insensitive comparison key

    // Keep this name only if a case-insensitive match has not been seen yet
    if (!seenLowercase.has(key)) {
      seenLowercase.add(key);
      uniqueNames.push(name);
    }
}

    // Sorts alphabethically, case-insensitive
    uniqueNames.sort((a, b) => a.toLowerCase().localeCompare(b.toLowerCase()));
    return uniqueNames;
}

// Test list with duplicates and inconsistent capitalisation
const testNames = ["Alice", "bob", "alice", "Charlie", "BOB", "dave", "Eve", "charlie"];

console.log("Original list:", testNames);
console.log("Processed list:", processNames(testNames));
```

```
Overwriting cleanNames_Modified.js
```

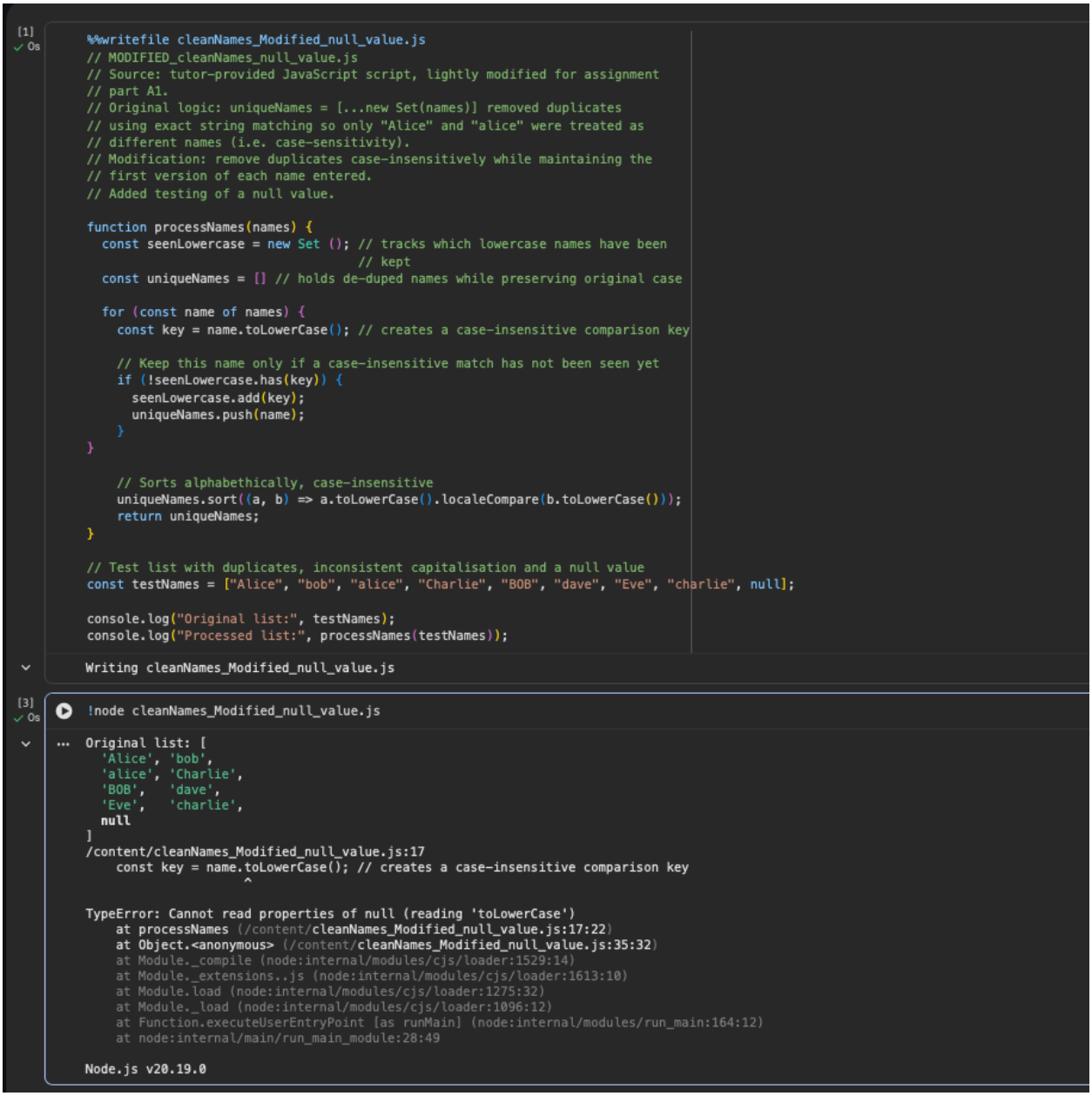
```
!node cleanNames_Modified.js
```

```
Original list: [
  'Alice', 'bob',
  'alice', 'Charlie',
  'BOB',   'dave',
  'Eve',   'charlie'
]
Processed list: [ 'Alice', 'bob', 'Charlie', 'dave', 'Eve' ]
```

Link to evidence: https://colab.research.google.com/drive/1HGEwZGJHqE-RjB2zKsNuv9lXFiueo9t3?usp=share_link

Appendix 5: Modified JavaScript code with `null`

The script was modified to become case-insensitive and a `null` value was added to the test data.



```
[1]
✓ 0s
%%writefile cleanNames_Modified_null_value.js
// MODIFIED_cleanNames_null_value.js
// Source: tutor-provided JavaScript script, lightly modified for assignment
// part A1.
// Original logic: uniqueNames = [...new Set(names)] removed duplicates
// using exact string matching so only "Alice" and "alice" were treated as
// different names (i.e. case-sensitivity).
// Modification: remove duplicates case-insensitively while maintaining the
// first version of each name entered.
// Added testing of a null value.

function processNames(names) {
  const seenLowercase = new Set (); // tracks which lowercase names have been
                                    // kept
  const uniqueNames = [] // holds de-duped names while preserving original case

  for (const name of names) {
    const key = name.toLowerCase(); // creates a case-insensitive comparison key

    // Keep this name only if a case-insensitive match has not been seen yet
    if (!seenLowercase.has(key)) {
      seenLowercase.add(key);
      uniqueNames.push(name);
    }
}

    // Sorts alphabethically, case-insensitive
    uniqueNames.sort((a, b) => a.toLowerCase().localeCompare(b.toLowerCase()));
    return uniqueNames;
}

// Test list with duplicates, inconsistent capitalisation and a null value
const testNames = ["Alice", "bob", "alice", "Charlie", "BOB", "dave", "Eve", "charlie", null];

console.log("Original list:", testNames);
console.log("Processed list:", processNames(testNames));

Writing cleanNames_Modified_null_value.js

[3]
✓ 0s
!node cleanNames_Modified_null_value.js

Original list: [
  'Alice', 'bob',
  'alice', 'Charlie',
  'BOB',   'dave',
  'Eve',   'charlie',
  null
]
/content/cleanNames_Modified_null_value.js:17
    const key = name.toLowerCase(); // creates a case-insensitive comparison key
                     ^

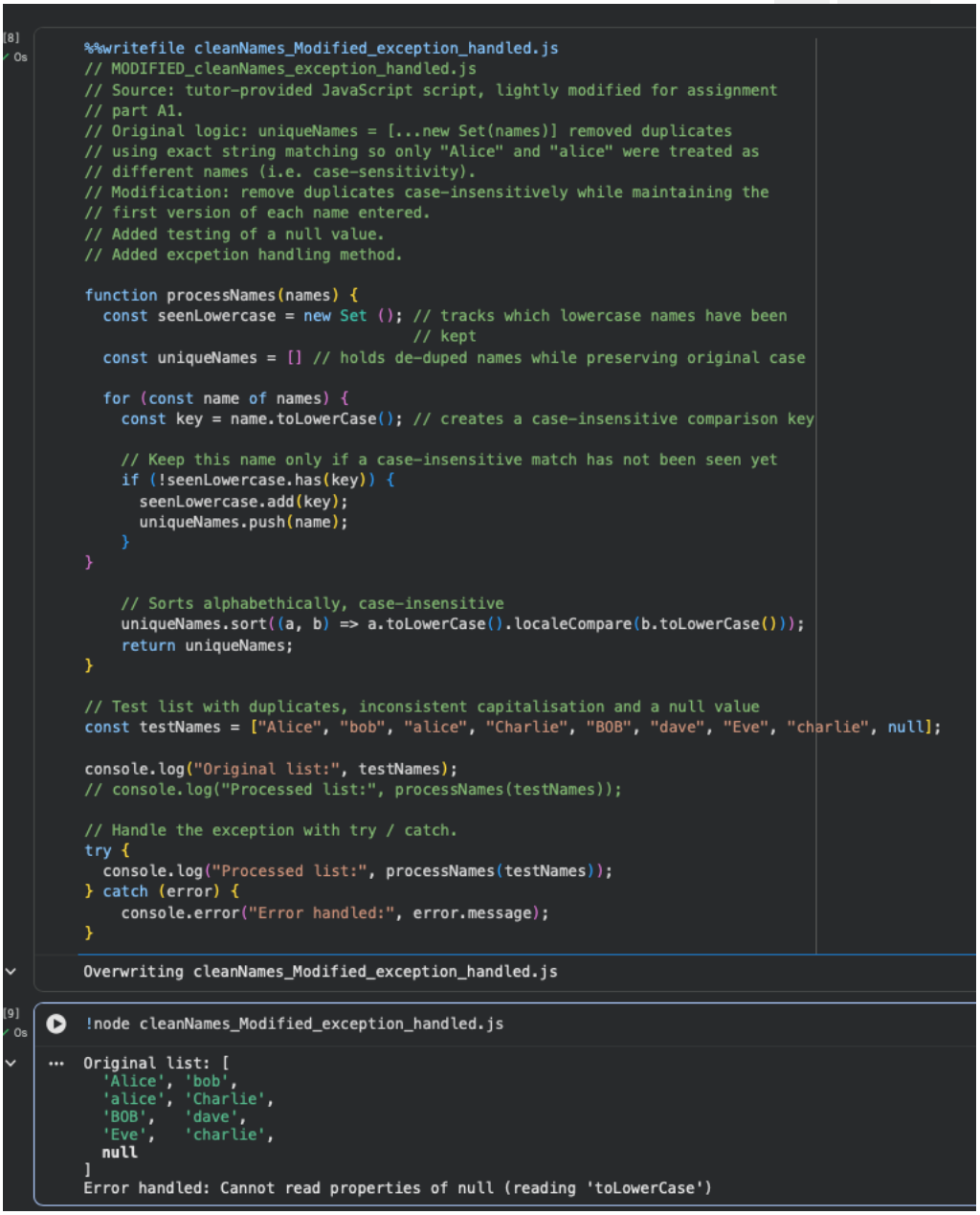
TypeError: Cannot read properties of null (reading 'toLowerCase')
    at processNames (/content/cleanNames_Modified_null_value.js:17:22)
    at Object.<anonymous> (/content/cleanNames_Modified_null_value.js:35:32)
    at Module._compile (node:internal/modules/cjs/loader:1529:14)
    at Module._extensions..js (node:internal/modules/cjs/loader:1613:10)
    at Module.load (node:internal/modules/cjs/loader:1275:32)
    at Module._load (node:internal/modules/cjs/loader:1096:12)
    at Function.executeUserEntryPoint [as runMain] (node:internal/modules/run_main:164:12)
    at node:internal/main/run_main_module:28:49

Node.js v20.19.0
```

Link to evidence: https://colab.research.google.com/drive/1ppSWsCcdp-NFpEUPy-6FFdKAxw201lm1?usp=share_link

Appendix 6: Modified JavaScript code with exception handling

The script was modified to become case-insensitive, a `null` value was added to the test data, and the resulting exception was handled using `try`/`catch`.



```
%%writefile cleanNames_Modified_exception_handled.js
// MODIFIED_cleanNames_exception_handled.js
// Source: tutor-provided JavaScript script, lightly modified for assignment
// part A1.
// Original logic: uniqueNames = [...new Set(names)] removed duplicates
// using exact string matching so only "Alice" and "alice" were treated as
// different names (i.e. case-sensitivity).
// Modification: remove duplicates case-insensitively while maintaining the
// first version of each name entered.
// Added testing of a null value.
// Added excpetion handling method.

function processNames(names) {
  const seenLowercase = new Set (); // tracks which lowercase names have been
                                    // kept
  const uniqueNames = [] // holds de-duped names while preserving original case

  for (const name of names) {
    const key = name.toLowerCase(); // creates a case-insensitive comparison key

    // Keep this name only if a case-insensitive match has not been seen yet
    if (!seenLowercase.has(key)) {
      seenLowercase.add(key);
      uniqueNames.push(name);
    }
}

    // Sorts alphabethically, case-insensitive
    uniqueNames.sort((a, b) => a.toLowerCase().localeCompare(b.toLowerCase()));
    return uniqueNames;
}

// Test list with duplicates, inconsistent capitalisation and a null value
const testNames = ["Alice", "bob", "alice", "Charlie", "BOB", "dave", "Eve", "charlie", null];

console.log("Original list:", testNames);
// console.log("Processed list:", processNames(testNames));

// Handle the exception with try / catch.
try {
  console.log("Processed list:", processNames(testNames));
} catch (error) {
    console.error("Error handled:", error.message);
}
```

```
Overwriting cleanNames_Modified_exception_handled.js
```

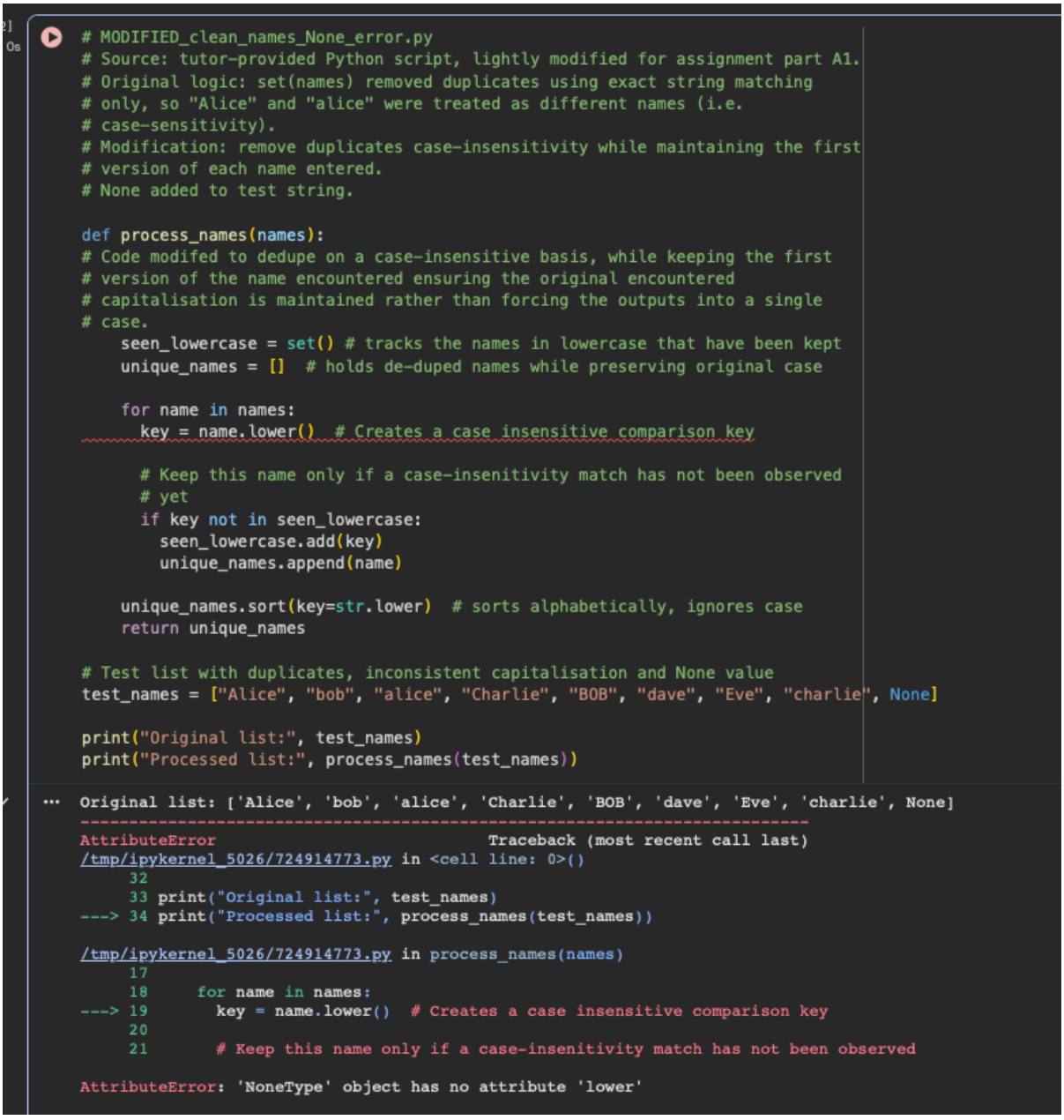
```
!node cleanNames_Modified_exception_handled.js
```

```
Original list: [
  'Alice', 'bob',
  'alice', 'Charlie',
  'BOB',   'dave',
  'Eve',   'charlie',
  null
]
Error handled: Cannot read properties of null (reading 'toLowerCase')
```

Link to evidence: https://colab.research.google.com/drive/12aZMnI6F13VylHYtOgnAPlvJAJEwm4Yi?usp=share_link

Appendix 7: Modified Python Script with `None`

The script was modified to become case-insensitive, and a `None` value was added to the test data.



```
# MODIFIED_clean_names_None_error.py
# Source: tutor-provided Python script, lightly modified for assignment part A1.
# Original logic: set(names) removed duplicates using exact string matching
# only, so "Alice" and "alice" were treated as different names (i.e.
# case-sensitivity).
# Modification: remove duplicates case-insensitivity while maintaining the first
# version of each name entered.
# None added to test string.

def process_names(names):
# Code modifed to dedupe on a case-insensitive basis, while keeping the first
# version of the name encountered ensuring the original encountered
# capitalisation is maintained rather than forcing the outputs into a single
# case.
    seen_lowercase = set() # tracks the names in lowercase that have been kept
    unique_names = []  # holds de-duped names while preserving original case

    for name in names:
      key = name.lower()  # Creates a case insensitive comparison key

      # Keep this name only if a case-insenitivity match has not been observed
      # yet
      if key not in seen_lowercase:
        seen_lowercase.add(key)
        unique_names.append(name)

    unique_names.sort(key=str.lower)  # sorts alphabetically, ignores case
    return unique_names

# Test list with duplicates, inconsistent capitalisation and None value
test_names = ["Alice", "bob", "alice", "Charlie", "BOB", "dave", "Eve", "charlie", None]

print("Original list:", test_names)
print("Processed list:", process_names(test_names))
```

```
Original list: ['Alice', 'bob', 'alice', 'Charlie', 'BOB', 'dave', 'Eve', 'charlie', None]
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
/tmp/ipykernel_5026/724914773.py in <cell line: 0>()
     32
     33 print("Original list:", test_names)
---> 34 print("Processed list:", process_names(test_names))

/tmp/ipykernel_5026/724914773.py in process_names(names)
     17
     18     for name in names:
---> 19       key = name.lower()  # Creates a case insensitive comparison key
     20
     21       # Keep this name only if a case-insenitivity match has not been observed

AttributeError: 'NoneType' object has no attribute 'lower'
```

Link to evidence: https://colab.research.google.com/drive/1wnGm9Ak_8y6EeEjuFGrBp0GYoWl_rV2h?usp=share_link

Appendix 8: Modified Python Script with exception handling

The script was modified to become case-insensitive, a `None` value was added to the test data, and the resulting exception was handled using `try`/`except`/`else`/`finally`.

```
# MODIFIED_clean_names_excpetion_handled.py
# Source: tutor-provided Python script, lightly modified for assignment part A1.
# Original logic: set(names) removed duplicates using exact string matching
# only, so "Alice" and "alice" were treated as different names (i.e.
# case-sensitivity).
# Modification: remove duplicates case-insensitivity while maintaining the first
# version of each name entered.
# None added to test string.
# Error-handling test: includes a None value and handles it using try / except.

def process_names(names):
# Code modifed to dedupe on a case-insensitive basis, while keeping the first
# version of the name encountered ensuring the original encountered
# capitalisation is maintained rather than forcing the outputs into a single
# case.
    seen_lowercase = set() # tracks the names in lowercase that have been kept
    unique_names = []  # holds de-duped names while preserving original case

    for name in names:
      key = name.lower()  # Creates a case insensitive comparison key

      # Keep this name only if a case-insenitivity match has not been observed
      # yet
      if key not in seen_lowercase:
        seen_lowercase.add(key)
        unique_names.append(name)

    unique_names.sort(key=str.lower)  # sorts alphabetically, ignores case
    return unique_names

# Test list with duplicates, inconsistent capitalisation and None value
test_names = ["Alice", "bob", "alice", "Charlie", "BOB", "dave", "Eve", "charlie", None]

print("Original list:", test_names)
# print("Processed list:", process_names(test_names))#

# Handle the exception using try / except / else / finally
try:
    processed_names = process_names(test_names)

except AttributeError as error:
    print("Error handled:", error)

else:
    print("Processed list:", processed_names)

finally:
    print("Test completed.")
```

```
Original list: ['Alice', 'bob', 'alice', 'Charlie', 'BOB', 'dave', 'Eve', 'charlie', None]
Error handled: 'NoneType' object has no attribute 'lower'
Test completed.
```

Link to evidence: https://colab.research.google.com/drive/1dMzvMdg4DowSBXJNgAudZOAh99d6Z61y?usp=share_link